Architecture

Ingestion pipeline for fetching data from Belgium's eAIP

Summary

Every 28 days the Belgian AIP publisher (skeyes) reissues its aeronautical publication as HTML documents. This system fetches those bytes, extracts the facts and operations a controller needs (such as airport data, runway declared distances, navigation aids, etc.) and attaches to every value the section, effective date and the verbatim source text it came from. A controller can open `EBBR AB 2.13` and see `LDA 3210 M`, and check the paragraph the publisher printed. When the next cycle lands, the system reports what changed at the field level and refuses to publish a cycle that fails its safety checks.

In the current version of the system, as of writing (commit `0cf9a4b`), there are **663** aeronautical features present in the database, with **422** having some geometry, and **241** without. The data being shown on the live web console aip.ishu.foo uses the cycle `2026-10-01-current` with `AMDT 010/2026` effective `2026-10-01`.

Please refer to the DECISION_LOG for the detailed Architectural Decision Record. For a visual overview, please refer to the README.

Pipeline

A piece of data takes eight stages to reach the console. The pipeline is Python because the parsing is HTML tree traversal and regex over tables, which `bs4` and `lxml` do directly, and a batch CLI does not care about interpreter startup:

1. **Discovery**: `pipeline/ingest/discovery.py` probes the portal for previous, current and preview editions, identifying each by their AIRAC effective date.
2. **Acquisition**: `pipeline/ingest/crawler.py` and `manifest.py` fetch documents byte-for-byte at 1 req/s and record URL, size, and SHA-256 per file in an immutable `manifest.jsonl`.
3. **Parsing**: `pipeline/parse/*` walks declarative table locators from `config/sections.yaml`, one parser per section family.
4. **Normalisation**: `pipeline/parse/converters.py` maps raw cell text to typed values and units, `pipeline/utils/hash_utils.py` hashes normalised values and units only.
5. **Validation**: `pipeline/validate/rules.py` applies `config/validation.yaml` and emits `validationIssues` rather than exceptions.
6. **Measuring diffs**: `pipeline/diff/engine.py` joins two cycles on natural key and classifies NEW / MODIFIED / DELETED / UNCHANGED with field-level change records.
7. **Promotion or quarantine**: `pipeline/load/load.py` stages the cycle, evaluates the promotion gate, then promotes it or persists it as `rejected` with its issues attached.
8. **Serve and export**: `api/server.ts` serves the loaded cycle read-only; `pipeline export` writes the same data to `data/export/<cycle>/` for database-free review.

Key highlights of the design

Four decisions carry most of the design.

Ingestion and the manifest

Ingestion uses a byte-for-byte fetch with a polite rate limit. The crawler is polite by nature and respects `robots.txt`. A live fetch is currently refused by `robots.txt` on skeyes's website; but this won't make the system not function. This was accounted for when I implemented the `ingest fetch --from-zip` CLI flag in the ingestion pipeline, so one can download the offline package (eAIP_Package.zip, 1.57 GB) and my pipeline can parse it perfectly.

This, in fact, is the only "authorised" way of fetching data right now, in this system.

Each document also carries an SHA-256 in `manifest.jsonl`. This is reproducibility machinery: an operator running the parse on their machine and me running it on mine must be working from identical bytes, and a per-document hash is what makes that checkable rather than assumed.

Parsing

This was an interesting decision. How does one ensure that a system such as this is scalable? How can one make sure that a web scraper, that is deeply tied to the nature of the data on a publisher's website, should in fact be somewhat "agnostic" to it?
The answer is *declarative configuration*, the same idea as Niri or Terraform: the configuration is the only source of truth, and the code is agnostic to what the website actually contains. A set of `.yaml` files in `config/` drives the rest of the system. There are but 19 section specifications in `config/sections.yaml` with declarative table locators, and provenance is carried per field (source of data). What happens when one day skeyes decides, "let's make that data on this page now instead of the previous one". How does my system survive? That's the interesting part - one can simply edit the config here and call it a day, as the rest of the code is truly agnostic to the publisher's website. Want to add more airports, a new section, or even change the publisher? Just edit the configs!

Validation

Eight rules run over every parsed feature, three of them format checks expressed as regular expressions. Each is a check the source material permits but does not enforce, so the pipeline enforces it: coordinate ranges, an operational bounding box derived from the Brussels FIR, natural-key format, natural-key uniqueness within a cycle, vertical-limit units, and three declared-distance relations. The rules, again, are not Python. Patterns and thresholds live in `config/validation.yaml`. Again, if any change was made to the publisher's webiste, we can simply edit the regex rules here instead of re-designing a new system.

Data models and diffing

Every "real world" thing is a `Feature` in this system, for ex. aerodromes, runways, waypoints, upper routes, restricted areas, etc. These have a common model as defined in `pipeline/models/feature.py`.
Natural keys are of interest here - they decide the identity of a feature and encode the hierarchy, for ex. `EBBR -> EBBR/RWY/01-19`. The natural key can, and it is the reason the diff works across cycles at all. It also fixes what a change means: because identity is the string, a renamed key is `DELETED` plus `NEW`, never `MODIFIED`. A *differently-named* feature is a *different* feature.
The content hash decides whether the fields have changed. `compute_content_hash` is an SHA-256 over canonical JSON. Provenance is deliberately excluded because every field carries a UTC timestamp of the fetch and it changes on every crawl by definition. Hashing it would make every feature appear as `MODIFIED` on each new fetch!

Promotion gate

A cycle is publishable only if it produces zero errors, no more than 100 warnings, and loses no runways and no more than 5% of features relative to the previously promoted cycle. All four thresholds live in `config/validation.yaml`. A cycle that fails is written to the database with status `rejected` and is not promoted, because a discarded failure leaves no evidence of what the publisher actually sent.

Known limits

The system can still be considered a *prototype*, not a "finished product". As such, there are some limitations. These are also mentioned in GitHub Issues, but to summarise just a few:

1. The cycle id is order-dependent. `discovery.py` appends the edition role only on a date collision, resolved by iteration order over `cycle_folders`. So current and previous share 2026-10-01 and whichever is probed first keeps the bare date. The committed cache, database and export are keyed 2026-10-01-current; ingest cycles today says 2026-10-01. These are the SAME edition with DIFFERENT names. It's in ADR 2 with both candidate fixes and the reason neither was applied: renaming orphans the cache, the database and the export, and the raw bytes can't be re-fetched.
2. Airspace has no geometry. 241 of 663 features are non-drawable by decision, because boundaries are prose and sovereign borders. The map shows less than the dataset contains.
3. There is no second cycle to diff against. One genuine comparison, NEW 1 / UNCHANGED 663. The MODIFIED and DELETED paths have never run against real data.